🎭 TEATRO · VENTA DE ENTRADAS EN LÍNEA

Guía de instalación, pruebas y puesta en producción

Solución a la prueba técnica PHP + MySQL. Esta guía explica, paso a paso, cómo instalar la aplicación, cómo comprobar que cumple cada punto del enunciado y cómo dejarla funcionando en un servidor de producción.

Probar en un equipo

XAMPP en Windows.
Unos 10 minutos.
Sección 3.

Producción con Docker

Un solo comando en cualquier servidor.
Unos 10 minutos.
Sección 4.

Producción con Apache o Nginx

Servidor Linux que ya tiene PHP y MySQL.
Unos 20 minutos.
Sección 5.

Octubre de 2026 · Acompaña al archivo LEAME.md de la entrega, que resume lo mismo en formato de texto.

Contenido

1. Qué contiene la entrega
2. Requisitos
3. Instalación para probar en un equipo (XAMPP)
4. Producción con Docker (recomendada)
5. Producción en un servidor Linux con Apache o Nginx
6. Datos de prueba
7. Cómo probar la solución, paso a paso
8. Configuración
9. Solución de problemas
10. Cómo funciona por dentro
11. Lista de comprobación para producción

Resumen del enunciado y dónde se cumple

Lo que pide el enunciado	Cómo se resolvió
Base de datos `TEATRO` con las tablas `OBRA` y `VENTAS` y sus campos	`database/teatro.sql` : crea el esquema, las tablas con tipos de datos elegidos con criterio, claves, índices y restricciones, y carga datos de prueba.
Página con las obras disponibles (con entradas y con fecha mayor que la actual)	`index.php` (la primera página se llama *index*).
Al elegir una obra se pide el identificador y se genera la venta	`comprar.php`
Número de entrada autoincremental que se muestra al usuario	`VENTAS.numero_venta` es `AUTO_INCREMENT` ; `confirmacion.php` lo muestra.
`VENTAS` se actualiza con cada compra y `OBRA` resta una disponible	Ambas cosas en una sola transacción (`includes/obras.php`), sin posibilidad de vender por encima del aforo.
Respaldo de la base de datos	`database/teatro.sql`
Archivo LÉAME con los pasos adicionales	`LEAME.md` y esta guía.
Ambiente de producción, sin ningún tipo de error	Manejo de errores sin detalles técnicos en pantalla, validaciones en el servidor, protección contra compras masivas, configuración de producción lista (Docker, Apache o Nginx) y 131 pruebas automáticas más 89 comprobaciones de instalación, todas superadas con PHP 8.1 y 8.5.

1. Qué contiene la entrega

La entrega es una carpeta llamada `teatro` (o el archivo `teatro.zip`, que la contiene). Su contenido:

Archivo o carpeta	Para qué sirve
`index.php`	Página principal: lista de obras disponibles.
`comprar.php`	Muestra la obra elegida, pide el identificador y registra la venta.
`confirmacion.php`	Muestra el número de entrada y los datos de la compra.
`consultar.php`	Permite volver a ver una entrada desde cualquier navegador, con su número y el identificador del comprador.
`assets/css/estilos.css`	Hoja de estilos. La aplicación no usa JavaScript.
`includes/`	Código interno (conexión, consultas, sesión, límites, plantillas). No se abre desde el navegador.
`includes/config.local.example.php`	Plantilla para indicar otra contraseña de MySQL u otros límites.
`database/teatro.sql`	Respaldo de la base de datos `TEATRO` con datos de prueba.
`docker-compose.yml`, `docker/`, `.env.example`	Instalación de producción con Docker (sección 4).
`LEAME.md`	Instrucciones en texto (resumen de esta guía).
`GUIA-INSTALACION.pdf`	Esta guía.

Tres formas de instalar

Forma	Cuándo usarla	Qué hay que tener	Sección
XAMPP	Revisar la solución en un equipo con Windows.	XAMPP con PHP 8.1 o superior.	3
Docker recomendada	Dejarla en producción con el menor trabajo posible.	Un servidor con Docker. Nada más: PHP, MySQL y el servidor web vienen en contenedores.	4
Apache o Nginx	Un servidor Linux que ya tiene PHP y MySQL.	PHP 8.1+, MySQL 8.0.16+ o MariaDB 10.3+, Apache o Nginx.	5

2. Requisitos

Componente	Versión y detalles
PHP	**8.1 o superior.** La aplicación se probó completa con PHP 8.1 y PHP 8.5. Necesita la extensión `pdo_mysql`, que viene activada en XAMPP, WAMP y Laragon.
Base de datos	**MySQL 8.0.16 o superior**, o **MariaDB 10.3 o superior**. XAMPP trae MariaDB, que sirve. Desarrollado y probado con MySQL 8.0.
Servidor web	Apache (por ejemplo, el de XAMPP) o Nginx con PHP-FPM.
Navegador	Cualquiera actual, con las cookies habilitadas (la compra usa una cookie de sesión).
Docker (opcional)	Docker Engine 24 o superior con `docker compose` (o Docker Desktop en Windows y macOS). Reemplaza todo lo anterior.

> **¿Por qué no MySQL 5.7?** La base de datos usa restricciones `CHECK` para que la propia base impida datos imposibles, como tener más entradas disponibles que el aforo. MySQL 5.7 acepta esas restricciones pero no las aplica (se aplican desde MySQL 8.0.16; MariaDB las aplica desde la versión 10.2). Además, MySQL 5.7 dejó de recibir actualizaciones en octubre de 2023.

Cómo saber qué versiones tiene

Qué	XAMPP (Windows, Símbolo del sistema)	Linux
PHP	`C:\xampp\php\php.exe -v`	`php -v`
Extensión pdo_mysql	`C:\xampp\php\php.exe -m | findstr pdo_mysql`	`php -m | grep pdo_mysql`
MySQL o MariaDB	`C:\xampp\mysql\bin\mysql.exe --version`	`mysql --version`

3. Instalación para probar en un equipo (XAMPP)

Este camino sirve para revisar la solución en un equipo con Windows. Si ya tiene XAMPP instalado, empiece en el paso 2.

1. **Instalar XAMPP**

   Descargar XAMPP para Windows desde `https://www.apachefriends.org` (una versión con PHP 8.1 o superior) e instalarlo con las opciones por defecto en `C:\xampp`.

2. **Iniciar Apache y MySQL**

   Abrir el **XAMPP Control Panel** y pulsar **Start** en **Apache** y en **MySQL**. Los dos nombres deben quedar con fondo verde.

3. **Copiar la aplicación**

   Descomprimir `teatro.zip` (o copiar la carpeta `teatro`) dentro de `C:\xampp\htdocs\`. Debe quedar así:

   ```
   C:\xampp\htdocs\teatro\index.php
   C:\xampp\htdocs\teatro\comprar.php
   C:\xampp\htdocs\teatro\database\teatro.sql
   ...
   ```

   Si al descomprimir queda una carpeta dentro de otra (`htdocs\teatro\teatro\index.php`), mueva la interior un nivel arriba.

4. **Restaurar la base de datos con phpMyAdmin**

   1. Abrir `http://localhost/phpmyadmin` en el navegador.
   2. Pulsar la pestaña **Importar** (arriba).
   3. En **Archivo a importar**, pulsar **Seleccionar archivo** y elegir `C:\xampp\htdocs\teatro\database\teatro.sql`.
   4. Dejar el resto de opciones como están y pulsar **Importar** (o **Continuar**), al final de la página.

   Resultado esperado: un mensaje verde de importación exitosa y, en la columna de la izquierda, la base `teatro` con las tablas `obra` y `ventas`.

   No hace falta crear la base antes: el script la crea. En Windows, MariaDB muestra los nombres en minúsculas; la aplicación funciona igual.

5. **(Alternativa al paso 4) Restaurar desde la consola**

   Abrir el **Símbolo del sistema** (CMD, no PowerShell) y ejecutar:

   ```
   C:\xampp\mysql\bin\mysql.exe -u root < C:\xampp\htdocs\teatro\database\teatro.sql
   ```

   Si su usuario `root` tiene contraseña, agregue `-p` y escríbala cuando la pida.

6 Configurar la conexión (solo si root tiene contraseña)

La aplicación se conecta a `localhost:3306` con el usuario `root` sin contraseña, que es lo que trae XAMPP. **Si no cambió esa contraseña, salte este paso.** Si la cambió:

1. Copiar `includes\config.local.example.php` con el nombre `includes\config.local.php`.
2. Abrirlo con el Bloc de notas y escribir la contraseña:

```
return [
    'db' => [
        'host'     => 'localhost',
        'port'     => 3306,
        'user'     => 'root',
        'password' => 'su_contraseña',
    ],
];
```

7 Abrir la aplicación

Ir a **`http://localhost/teatro/`**.

Resultado esperado: la página «Obras disponibles» con 5 obras.

8 Comprobar que los archivos internos están protegidos

Abrir `http://localhost/teatro/database/teatro.sql` y `http://localhost/teatro/includes/config.php`.

Resultado esperado: «Forbidden» / «Acceso prohibido» (error 403) en los dos.

XAMPP aplica los archivos `.htaccess` que traen las carpetas `includes/`, `database/` y `docker/`.

4. Producción con Docker (recomendada)

Con Docker, un solo comando levanta tres servicios ya configurados para producción: el servidor web (Nginx), PHP y MySQL. No hace falta instalar ni configurar PHP, MySQL ni Apache en el servidor.

Qué queda funcionando

Servicio	Qué hace
`web` (Nginx)	Es lo único accesible desde fuera (puerto 8080 por defecto). Solo ejecuta las cuatro páginas públicas y sirve la hoja de estilos; cualquier otra ruta (`includes/` , `database/` , `.env` ...) responde 404. Limita las solicitudes por IP.
`php` (PHP 8.5-FPM)	Configuración de producción: los errores van al log, nunca a la pantalla. Sistema de archivos de solo lectura.
`db` (MySQL 8.4)	Sin puertos publicados: solo el servicio `php` puede conectarse, por una red interna sin salida a Internet. La aplicación usa el usuario `teatro_app` , que solo puede leer, insertar, actualizar y borrar filas de `TEATRO` . La contraseña de `root` es aleatoria.

Los tres servicios se reinician solos si fallan o si se reinicia el servidor. Los datos se guardan en un volumen de Docker y se conservan entre reinicios.

4.1 Instalar Docker (si el servidor no lo tiene)

Sistema	Cómo
Ubuntu 22.04 o 24.04	`sudo apt update && sudo apt install -y docker.io docker-compose-v2`
Otras distribuciones Linux	Seguir `https://docs.docker.com/engine/install/`
Windows o macOS	Instalar **Docker Desktop** desde `https://www.docker.com/products/docker-desktop/` y abrirlo.

Para comprobar la instalación: `docker compose version` debe mostrar una versión.

4.2 Levantar la aplicación

1. **Copiar la aplicación al servidor**

   Por ejemplo, en `/opt/teatro` . Desde el equipo donde está el ZIP:

   ```
   scp teatro.zip usuario@IP_DEL_SERVIDOR:/tmp/
   ssh usuario@IP_DEL_SERVIDOR
   sudo apt install -y unzip
   sudo unzip /tmp/teatro.zip -d /opt/        # crea /opt/teatro
   ```

2 **Revisar la configuración (recomendado)**

```
cd /opt/teatro
sudo cp .env.example .env
sudo nano .env
```

Cambie al menos `TEATRO_DB_PASSWORD` por una clave propia (solo letras, números, puntos y guiones). Las demás opciones están explicadas en la sección 8. **Hágalo antes del paso 3**: la clave se fija la primera vez que se crea la base.

3 **Levantar los servicios**

```
cd /opt/teatro
sudo docker compose up -d
```

La primera vez tarda uno o dos minutos: descarga las imágenes, prepara PHP, crea la base `TEATRO` e importa `database/teatro.sql`.

4 **Comprobar que está en marcha**

```
sudo docker compose ps
```

Resultado esperado: los servicios `web`, `php` y `db` en estado «running» o «Up», y `db` como «healthy».

5 **Abrir el puerto en el cortafuegos (si lo hay)**

```
sudo ufw allow 8080/tcp
```

6 **Abrir la aplicación**

Ir a `http://IP_DEL_SERVIDOR:8080/` (o `http://localhost:8080/` si Docker está en el mismo equipo).

Resultado esperado: «Obras disponibles» con 5 obras. Y `http://IP_DEL_SERVIDOR:8080/database/teatro.sql` responde «404 Not Found».

> Si en la carpeta existe `includes/config.local.php` (por ejemplo, de una prueba anterior con XAMPP), bórrelo: tiene prioridad sobre la configuración de Docker y haría que la aplicación busque MySQL en otro lugar.

4.3 Publicarla con un dominio y HTTPS

Para producción real conviene un dominio con certificado HTTPS. La forma más simple es poner **Caddy** delante, que obtiene y renueva el certificado solo:

1. Apuntar el dominio (por ejemplo, `teatro.ejemplo.com`) a la IP del servidor, con un registro DNS de tipo A.
2. Hacer que el puerto 8080 solo se escuche dentro del servidor: en `.env`, poner `TEATRO_PUERTO=127.0.0.1:8080` y aplicar con `sudo docker compose up -d`.
3. Instalar Caddy y configurarlo:

```
sudo apt install -y caddy        # si apt no lo encuentra: https://caddyserver.com/docs/install
sudo nano /etc/caddy/Caddyfile
```

Reemplazar el contenido por:

```
teatro.ejemplo.com {
    reverse_proxy 127.0.0.1:8080
}
```

```
sudo systemctl reload caddy
sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
sudo ufw delete allow 8080/tcp      # el 8080 ya no se usa desde fuera
```

Al abrir `https://teatro.ejemplo.com` se verá la aplicación con candado. La aplicación reconoce el HTTPS por el encabezado `X-Forwarded-Proto` que envía el proxy, y marca la cookie de sesión como `Secure`. Nginx, dentro de Docker, toma la IP real del visitante de `X-Forwarded-For`, para que los límites por IP se apliquen a cada visitante y no al proxy. Si ya usa Nginx o Apache como proxy, sirve igual: deben reenviar a `http://127.0.0.1:8080` con esos dos encabezados.

4.4 Operación diaria

Para	Comando (desde `/opt/teatro`)
Ver el estado	`sudo docker compose ps`
Ver los errores de PHP y avisos de la aplicación	`sudo docker compose logs php`
Consultar la base de datos	`sudo docker compose exec db sh -c 'mysql -uteatro_app -p"$TEATRO_DB_PASSWORD" TEATRO'`
Respaldar la base de datos	`sudo docker compose exec -T db sh -c 'mysqldump --single-transaction --no-tablespaces -uteatro_app -p"$TEATRO_DB_PASSWORD" TEATRO' > respaldo.sql`
Detener (los datos se conservan)	`sudo docker compose down`
Volver a levantar	`sudo docker compose up -d`
Volver a los datos de prueba iniciales (borra las ventas)	`sudo docker compose down -v && sudo docker compose up -d`
Actualizar la aplicación a una versión nueva	Reemplazar los archivos de la carpeta (sin borrar `.env`) y ejecutar `sudo docker compose up -d --build`.

5. Producción en un servidor Linux con Apache o Nginx

Para un servidor que ya tiene (o tendrá) PHP y MySQL instalados directamente. Los ejemplos son para Ubuntu 22.04 o 24.04; en Debian, cambie `mysql-server` por `mariadb-server`.

5.1 Pasos comunes

1. **Instalar los paquetes**

```
# Con Apache:
sudo apt update && sudo apt install -y apache2 php libapache2-mod-php php-mysql mysql-server
unzip
# Con Nginx:
sudo apt update && sudo apt install -y nginx php-fpm php-mysql mysql-server unzip
```

Comprobar que PHP es 8.1 o superior con `php -v`.

2. **Copiar la aplicación**

```
# Apache: /var/www/html/teatro  (se abrirá en http://servidor/teatro/)
sudo unzip /tmp/teatro.zip -d /var/www/html/
# Nginx: /var/www/teatro  (se abrirá en la raíz del dominio)
sudo unzip /tmp/teatro.zip -d /var/www/
```

Los archivos pueden quedar con dueño `root`: el servidor web solo necesita leerlos.

3. **Restaurar la base de datos**

```
sudo mysql < /var/www/html/teatro/database/teatro.sql     # o /var/www/teatro/... con Nginx
```

4. **Crear el usuario de la aplicación con permisos mínimos**

En producción la aplicación no debe usar `root`. Este usuario solo puede leer, insertar, actualizar y borrar filas de `TEATRO`:

```
sudo mysql -e "CREATE USER 'teatro_app'@'localhost' IDENTIFIED BY 'una_clave_segura';
               GRANT SELECT, INSERT, UPDATE, DELETE ON TEATRO.* TO
'teatro_app'@'localhost';"
```

5. **Configurar la conexión**

```
cd /var/www/html/teatro        # o /var/www/teatro con Nginx
sudo cp includes/config.local.example.php includes/config.local.php
sudo nano includes/config.local.php   # 'user' => 'teatro_app', 'password' =>
'una_clave_segura'
sudo chown root:www-data includes/config.local.php
sudo chmod 640 includes/config.local.php   # solo el servidor web puede leer la clave
```

5.2 Con Apache

6 **Bloquear las carpetas internas**

Ubuntu y Debian ignoran los archivos `.htaccess` por defecto (`AllowOverride None`), así que el bloqueo se hace en la configuración de Apache. Crear el archivo:

```
sudo nano /etc/apache2/conf-available/teatro.conf
```

con este contenido:

```
<DirectoryMatch "^/var/www/html/teatro/(includes|database|docker)(/|$)">
    Require all denied
</DirectoryMatch>
<FilesMatch "^(\.env.*|docker-compose\.yml|.*\.md)$">
    Require all denied
</FilesMatch>
```

Activarlo y comprobar:

```
sudo a2enconf teatro && sudo systemctl reload apache2
curl -I http://localhost/teatro/database/teatro.sql
```

Resultado esperado: `HTTP/1.1 403 Forbidden` .

7 **HTTPS con un dominio**

```
sudo apt install -y certbot python3-certbot-apache
sudo certbot --apache
```

Certbot pide el dominio, obtiene el certificado y configura la redirección a HTTPS. La renovación es automática.

8 **Solo si Apache está detrás de un proxy o balanceador**

Para que los límites por IP vean la IP real del visitante y no la del proxy, activar `mod_remoteip` indicando la IP del proxy (aquí, `10.0.0.5` como ejemplo):

```
sudo a2enmod remoteip
printf 'RemoteIPHeader X-Forwarded-For\nRemoteIPTrustedProxy 10.0.0.5\n' | sudo tee
/etc/apache2/conf-available/remoteip.conf
sudo a2enconf remoteip && sudo systemctl reload apache2
```

El HTTPS detrás del proxy se detecta solo (encabezados `X-Forwarded-Proto` o `Forwarded`).

La aplicación queda en `https://dominio/teatro/` (o `http://IP_DEL_SERVIDOR/teatro/` sin dominio).

5.3 Con Nginx y PHP-FPM

Con Nginx no se usan los `.htaccess` . La configuración siguiente es una lista de lo único que se puede abrir; todo lo demás no existe para el navegador.

6 Crear el sitio

```
ls /run/php/                    # anote el nombre del socket, por ejemplo php8.3-fpm.sock
sudo nano /etc/nginx/sites-available/teatro
```

```
server {
    listen 80;
    server_name teatro.ejemplo.com;
    root /var/www/teatro;

    # Solo se ejecutan las páginas públicas.
    location ~ ^/(index|comprar|confirmacion|consultar)\.php$ {
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;    # el socket anotado arriba
    }

    location ^~ /assets/ { try_files $uri =404; }
    location = / { rewrite ^ /index.php last; }

    # Todo lo demás (includes/, database/, docker/, .env...) responde 404.
    location / { return 404; }
}
```

```
sudo ln -s /etc/nginx/sites-available/teatro /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default      # si este servidor es solo para Teatro
sudo nginx -t && sudo systemctl reload nginx
curl -I http://localhost/database/teatro.sql
```

Resultado esperado: `nginx -t` dice «syntax is ok» y la última línea muestra `HTTP/1.1 404 Not Found`.

7 HTTPS con un dominio

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx
```

6. Datos de prueba

`database/teatro.sql` carga 7 obras. Sus fechas se calculan **a partir del momento en que se importa**, así que siempre hay obras futuras para probar, sin importar el día en que se revise.

Cód.	Obra	Fecha	Aforo	Disponibles	¿Aparece en la lista?
1	La casa de Bernarda Alba	en 3 días, 20:00	120	120	Sí
2	Bodas de sangre	en 7 días, 19:30	80	78	Sí
3	La vida es sueño	en 10 días, 18:00	5	**2**	Sí, hasta que se vendan las 2
4	Don Juan Tenorio	en 14 días, 21:00	200	200	Sí
5	El sí de las niñas	en 21 días, 17:00	60	60	Sí
6	Luces de bohemia	en 5 días, 20:30	3	**0**	**No**: está agotada
7	Fuenteovejuna	**hace 2 días**	50	48	**No**: su fecha ya pasó

La tabla `VENTAS` trae 10 ventas previas (coherentes con las disponibles de cada obra), así que **la primera compra nueva genera la entrada número 11**.

Para volver al estado inicial en cualquier momento, basta con importar `database/teatro.sql` otra vez (en Docker: `docker compose down -v && docker compose up -d`).

7. Cómo probar la solución, paso a paso

Las direcciones usan `http://localhost/teatro/` (XAMPP). En Docker, use `http://localhost:8080/`; en Nginx, la raíz del dominio. Conviene empezar con la base recién importada, para que los números coincidan con los de esta guía.

7.1 Recorrido principal (lo que pide el enunciado)

	Acción	Resultado esperado
☐	**1.** Abrir `http://localhost/teatro/`.	Página «Obras disponibles» con **5 obras** ordenadas por fecha, cada una con fecha y hora, sala y entradas disponibles. *Luces de bohemia* (agotada) y *Fuenteovejuna* (fecha pasada) **no** aparecen.
☐	**2.** Pulsar «Comprar entrada» en *La vida es sueño*.	Se muestra la obra elegida (fecha, sala, «2 de 5» disponibles) y un campo **Identificador**.
☐	**3.** Escribir `V-12345678` y pulsar «Comprar entrada».	«¡Compra realizada con éxito!» con el **número de entrada 11**, la obra, la fecha, la sala, el comprador y la fecha de compra.
☐	**4.** Verificar la base de datos (phpMyAdmin → pestaña SQL, o la consola) con las dos consultas de abajo.	Hay una venta 11 de la obra 3 para `V-12345678`, y `disponibles` de la obra 3 bajó de 2 a 1.
☐	**5.** Volver a «Obras disponibles».	*La vida es sueño* muestra «1 de 5».
☐	**6.** Comprar la última entrada de *La vida es sueño* (cualquier identificador válido).	Se genera la entrada **12** y la obra **deja de aparecer** en la lista, porque ya no tiene entradas.

```
SELECT * FROM TEATRO.VENTAS ORDER BY numero_venta DESC LIMIT 1;
-- numero_venta 11, cod_obra 3, comprador V-12345678, fecha_compra = el momento de la compra

SELECT cod_obra, nombre, aforo, disponibles FROM TEATRO.OBRA WHERE cod_obra = 3;
-- disponibles = 1
```

7.2 Recuperar una entrada desde otro navegador

	Acción	Resultado esperado
☐	Abrir la aplicación en otro navegador o en una ventana privada y escribir en la barra `…/confirmacion.php?venta=11`.	«Entrada no encontrada»: las entradas no se ven cambiando el número en la dirección.
☐	Pulsar «Consultar mi entrada» (arriba a la derecha) y escribir `11` y `V-12345678`.	Se muestra la entrada 11 completa.
☐	Repetir con `11` y otro identificador, por ejemplo `V99999999`.	«No encontramos una entrada con ese número e identificador».

7.3 Errores controlados

	Acción	Resultado esperado
☐	Pulsar «Comprar entrada» con el identificador vacío.	«Ingrese su identificador para continuar». No se registra ninguna venta.
☐	Escribir `ab`, `<script>` o `1' OR '1'='1` como identificador.	Mensaje con el formato esperado (5 a 20 letras, números o guiones). No se registra ninguna venta y el texto ingresado se muestra tal cual, sin ejecutarse.
☐	Escribir `v-12.345.678` como identificador.	Se acepta y se guarda normalizado como `V-12345678`.
☐	Pulsar F5 en la página de confirmación (y aceptar si el navegador pregunta).	Se vuelve a mostrar la misma entrada. No se genera otra venta.
☐	En el formulario de compra, pulsar «Comprar entrada» varias veces seguidas.	Se registra **una sola** venta.
☐	Comprar en *Bodas de sangre* 4 entradas con el mismo identificador y luego intentar una quinta.	Las 4 primeras se registran; la quinta muestra «Este identificador ya tiene el máximo de 4 entradas permitidas para esta obra» y no se registra.
☐	Abrir `comprar.php?obra=6` (agotada), `?obra=7` (fecha pasada), `?obra=999` y `?obra=abc`.	«Obra no disponible» en todos los casos.
☐	Detener MySQL (en XAMPP, botón *Stop*; en Docker, `docker compose stop db`) y recargar la página.	«Servicio no disponible», sin ningún detalle técnico. Al volver a iniciar MySQL, todo funciona de nuevo.

7.4 Protección contra compras masivas

	Acción	Resultado esperado
☐	Hacer más de 20 compras en una hora desde el mismo equipo. Para comprobarlo rápido, baje antes el límite a 2 (`'compras_por_ip' => 2` en `config.local.php`, o `TEATRO_LIMITE_COMPRAS_IP=2` en `.env` y `docker compose up -d`; ver sección 8).	Al superar el límite, página «Demasiadas compras». No se registra la venta. Vuelva a dejar el límite como estaba al terminar.
☐	Hacer más de 10 consultas en «Consultar mi entrada» en 15 minutos.	A partir de la 11.ª, página «Demasiadas consultas».

> El límite por defecto (20 compras por hora) alcanza de sobra para todas las pruebas de esta guía. Para hacer más compras desde un mismo equipo, súbalo o desactívelo (con `0`) como se explica en la sección 8.

7.5 Seguridad

	Acción	Resultado esperado
☐	Abrir `…/database/teatro.sql` y `…/includes/config.php`.	403 (XAMPP y Apache) o 404 (Docker y Nginx). Nunca se descarga ni se muestra el archivo.

☐	Ver el código fuente de cualquier página (Ctrl+U).	No hay JavaScript ni datos internos; los textos de las obras aparecen escapados.

7.6 Lo que no se puede probar a mano (y cómo se probó)

Algunos escenarios necesitan muchas peticiones al mismo tiempo. Se probaron con pruebas automáticas, superadas con PHP 8.1 y PHP 8.5:

Escenario	Resultado comprobado
20 personas compran a la vez las 2 últimas entradas de una obra	Exactamente 2 ventas; las otras 18 reciben «Entradas no disponibles». Nunca se vende por encima del aforo.
El mismo formulario enviado 10 veces a la vez (doble clic con conexión lenta)	1 sola venta; las 10 respuestas llevan a la misma entrada.
El mismo identificador compra desde 5 navegadores a la vez	Exactamente 4 ventas (el máximo por persona); la quinta se rechaza.
MySQL bloqueado momentáneamente por otra operación	La compra se reintenta sola y se completa una sola vez.
MySQL caído o contraseña incorrecta	Página amigable sin detalles técnicos; el detalle queda solo en el log del servidor, sin la contraseña.
Después de todas las pruebas	En todas las obras se cumple `disponibles = aforo − ventas`, y los números de venta son correlativos.

8. Configuración

La aplicación funciona sin configurar nada con XAMPP. Para cambiar algo:

- **XAMPP, Apache o Nginx:** copiar `includes/config.local.example.php` como `includes/config.local.php` y escribir solo los valores que se quieran cambiar.
- **Docker:** copiar `.env.example` como `.env` y cambiar los valores. Después, `docker compose up -d`.

Opción (config.local.php)	Variable (.env / entorno)	Defecto	Qué hace
`db.host`, `db.port`	`TEATRO_DB_HOST`, `TEATRO_DB_PORT`	localhost, 3306	Dónde está MySQL.
`db.user`, `db.password`	`TEATRO_DB_USER`, `TEATRO_DB_PASSWORD`	root, vacía	Usuario y contraseña de MySQL. En Docker el usuario es siempre `teatro_app`.
`limites.entradas_por_comprador`	`TEATRO_LIMITE_ENTRADAS_COMPRADOR`	4	Entradas que una persona (identificador) puede comprar para cada obra.
`limites.compras_por_ip`	`TEATRO_LIMITE_COMPRAS_IP`	20	Compras por dirección IP en una hora.
`limites.consultas_por_ip`	`TEATRO_LIMITE_CONSULTAS_IP`	10	Consultas de entradas por dirección IP en 15 minutos.
`debug`	`TEATRO_DEBUG`	false	Muestra el detalle técnico de los errores. Solo para diagnosticar una instalación; **nunca en producción**.
—	`TEATRO_PUERTO`	8080	Solo Docker: puerto público. `127.0.0.1:8080` lo deja accesible solo desde el propio servidor (para usar un proxy con HTTPS).

—	`TEATRO_ZONA_HORARIA`	-06:00	Solo Docker: zona horaria de MySQL, que decide qué obras ya pasaron.

En los tres límites, el valor `0` lo desactiva. Ejemplo de `config.local.php` para hacer muchas compras de prueba con XAMPP:

```
<?php
return [
    'limites' => [
        'compras_por_ip' => 0,   // sin límite de compras por IP mientras se prueba
    ],
];
```

9. Solución de problemas

Síntoma	Causa probable y solución
«Servicio no disponible»	La aplicación no puede conectarse a MySQL. Verifique que MySQL esté iniciado, que se haya importado `database/teatro.sql` y que la contraseña sea la correcta (sección 3, paso 6). El detalle exacto queda en el log de PHP o del servidor web: en XAMPP, `C:\xampp\php\logs\php_error_log` o `C:\xampp\apache\logs\error.log`; en Linux, `/var/log/apache2/error.log` o `/var/log/nginx/error.log`; en Docker, `docker compose logs php`.
Página en blanco, o el navegador descarga el código PHP	El servidor web no está ejecutando PHP. Verifique que Apache esté iniciado y que la dirección empiece con `http://localhost/`: abrir el archivo directamente desde la carpeta no ejecuta PHP.
«No hay obras disponibles»	No se importaron los datos de prueba, o se importaron hace semanas y las fechas ya pasaron. Importe `database/teatro.sql` de nuevo.
«Su navegador no está aceptando cookies»	La compra y la consulta usan una cookie de sesión. Habilite las cookies para el sitio.
«Demasiadas compras» o «Demasiadas consultas»	Es la protección contra compras masivas. Espere (una hora o 15 minutos) o suba el límite (sección 8).
`database/teatro.sql` se descarga desde el navegador	El servidor no aplica el bloqueo. En Apache con `AllowOverride None`, siga el paso 6 de la sección 5.2; en Nginx, use la configuración de la sección 5.3.
Error `#1045 Access denied` al importar en phpMyAdmin o en la consola	El usuario `root` tiene contraseña. En consola, agregue `-p` y escríbala.
La importación falla en `CHECK` o en `utf8mb4`	La versión de MySQL o MariaDB es demasiado antigua (sección 2).
Docker: «Servicio no disponible» justo después de `docker compose up -d`	MySQL todavía está importando la base. Espere a que `docker compose ps` muestre `db` como «healthy» y recargue.
Docker: `port is already allocated`	El puerto 8080 está ocupado. Ponga otro en `.env` (`TEATRO_PUERTO=8081`) y ejecute `docker compose up -d`.
Docker: se cambió `TEATRO_DB_PASSWORD` después del primer arranque y aparece «Servicio no disponible»	La clave se fija al crear la base. Vuelva a crearla con `docker compose down -v && docker compose up -d` (borra las ventas registradas).

Para ver el detalle técnico de un error en pantalla mientras se diagnostica una instalación, agregue `'debug' => true,` en `includes/config.local.php`. **Quítelo al terminar.**

10. Cómo funciona por dentro

Recorrido

```
index.php  ──«Comprar entrada»──▶  comprar.php  ──identificador──▶  confirmacion.php
(obras disponibles)                (pide el identificador)          (número de entrada)

consultar.php  ──número + identificador──▶  confirmacion.php      (desde cualquier navegador)
```

Base de datos

Campo	Tipo	Motivo
`OBRA.cod_obra`	`INT UNSIGNED AUTO_INCREMENT`	Clave primaria, sin negativos.
`OBRA.nombre`	`VARCHAR(150) NOT NULL`	Títulos largos.
`OBRA.fecha_obra`	`DATETIME NOT NULL`	Una función tiene fecha **y hora**: deja de venderse cuando empieza.
`OBRA.aforo`, `disponibles`	`SMALLINT UNSIGNED NOT NULL`	Hasta 65 535 butacas en 2 bytes, sin negativos.
`OBRA.sala`	`VARCHAR(50) NOT NULL`	Nombre de la sala.
`VENTAS.numero_venta`	`INT UNSIGNED AUTO_INCREMENT`	Número de entrada autoincremental.
`VENTAS.cod_obra`	`INT UNSIGNED` + clave foránea	Toda venta corresponde a una obra existente; no se puede borrar una obra con ventas.
`VENTAS.comprador`	`VARCHAR(20) NOT NULL`	Identificador (documento) del comprador.
`VENTAS.fecha_compra`	`DATETIME DEFAULT CURRENT_TIMESTAMP`	Momento de la compra.

Además: restricciones `CHECK` (`aforo > 0`, `disponibles <= aforo`, nombre no vacío), índices para la lista de obras y para contar las entradas de cada comprador, motor InnoDB (transacciones) y `utf8mb4` para acentos y eñes.

Cómo se registra una venta sin vender de más

Todo ocurre dentro de una transacción:

```
-- 1. Descontar la entrada solo si todavía hay y la obra no ha empezado.
UPDATE OBRA SET disponibles = disponibles - 1
 WHERE cod_obra = ? AND disponibles > 0 AND fecha_obra > NOW();

-- 2. Si no modificó la fila: "Entradas no disponibles" y no se registra nada.
-- 3. Contar las entradas que ya tiene ese comprador para la obra (máximo 4).
SELECT COUNT(*) FROM VENTAS WHERE cod_obra = ? AND comprador = ? FOR UPDATE;

-- 4. Registrar la venta y confirmar. LAST_INSERT_ID() es el número de entrada.
INSERT INTO VENTAS (cod_obra, comprador, fecha_compra) VALUES (?, ?, NOW());
COMMIT;
```

El `UPDATE` bloquea la fila de la obra hasta el `COMMIT`, así que las compras simultáneas de una misma obra se atienden de a una: la última entrada se vende una sola vez y el límite por persona no se puede saltar comprando desde varios navegadores a la vez. Si MySQL aborta la transacción por un bloqueo (mucha carga), la compra se reintenta sola hasta 3 veces; como la transacción fallida se deshace completa, reintentar nunca duplica una venta.

Seguridad y prevención de errores

Riesgo	Cómo se previene
Errores técnicos en pantalla	Todo error se registra en el log y el usuario ve una página amigable, también si MySQL no responde. Los avisos de PHP se tratan como errores para no seguir en un estado inconsistente.
Inyección SQL	Todas las consultas usan sentencias preparadas (PDO, sin emulación).
XSS	Todo lo que se muestra pasa por `htmlspecialchars`; además, la política de seguridad de contenido (CSP) prohíbe ejecutar scripts.
Compra duplicada (doble clic, F5, Atrás)	Cada formulario lleva un token de un solo uso; después de comprar se redirige a la confirmación (POST/Redirect/GET).
CSRF (compras desde otro sitio)	El mismo token, más la cookie `SameSite=Lax`.
Bots que compran todas las entradas	Máximo 4 entradas por identificador y obra (en la misma transacción de la venta), máximo 20 compras por IP en una hora y, en Docker, límite de solicitudes por segundo en Nginx.
Ver o adivinar entradas ajenas	La confirmación solo muestra las ventas de esa sesión. Desde otro navegador se necesitan el número **y** el identificador, con un máximo de 10 consultas por IP cada 15 minutos.
Descarga de archivos internos	Bloqueo en la configuración del servidor web (sección 5) y, en Apache, también con `.htaccess`.
Robo de la cookie de sesión	Cookie `HttpOnly` y `SameSite=Lax`; `Secure` cuando el visitante llega por HTTPS, también detrás de un proxy o balanceador.
Clickjacking, rastreo de tipos, caché	Encabezados `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy` y `Cache-Control: no-store`.
Zona horaria	La regla «fecha mayor que la actual» la evalúa MySQL con `NOW()`, no PHP.

Alcance de la protección contra bots: los límites frenan a un script que compra desde una misma conexión o con un mismo identificador. Un ataque distribuido (miles de IP e identificadores inventados) solo se detiene verificando la identidad del comprador, con cuentas de usuario, pago o un CAPTCHA, que quedan fuera de lo que pide el enunciado.

11. Lista de comprobación para producción

	Comprobación
☐	PHP 8.1 o superior con `pdo_mysql`, y MySQL 8.0.16+ o MariaDB 10.3+ (o Docker).
☐	La aplicación usa un usuario de MySQL con permisos mínimos (`teatro_app`), no `root`.
☐	La contraseña de la base es propia (no la del ejemplo) y `config.local.php` o `.env` no se puede leer desde el navegador.
☐	`…/database/teatro.sql` responde 403 o 404.
☐	`debug` está desactivado (es el valor por defecto).
☐	El sitio responde por HTTPS con un certificado válido.
☐	Si hay un proxy delante, la aplicación ve la IP real del visitante (Docker lo hace solo; en Apache, `mod_remoteip`).
☐	Los límites contra compras masivas tienen valores adecuados para el negocio (sección 8).
☐	El cortafuegos solo deja pasar los puertos necesarios (80 y 443, o el de Docker).
☐	Hay un respaldo periódico de la base de datos (por ejemplo, el comando de la sección 4.4 en una tarea programada).
☐	El recorrido de la sección 7.1 funciona de punta a punta en el servidor de producción.